MS&E 228 (Winter 2026) — Lecture 9 Notes (Student Handout)

Double Machine Learning in Practice

Vasilis Syrgkanis
Stanford University

February 8, 2026

Readings. *Applied Causal Inference Powered by ML and AI*, §5, 6. Relevant papers: Chernozhukov et al. (2018), Bach et al. (2024).
Jupyter Notebooks 1. Simple Synthetic Price Elasticities. 2. Real World Price Elasticities in Panel Data

1 Introduction: From Theory to Practice

This lecture completes our theoretical discussion of Double Machine Learning (DML) and transitions to its practical application. We formalize the full DML algorithm with cross-fitting, establish its asymptotic normality, and discuss the necessary regularity conditions for valid inference. A key focus is the comparison between DML and the Doubly Robust (DR) estimator, clarifying when to use each. The main part of the lecture is a hands-on case study applying DML to estimate the price elasticity of demand from a real-world panel dataset of Amazon product sales. This involves tackling practical challenges like endogeneity from unobserved quality, non-independent observations requiring clustered standard errors, and data leakage in cross-fitting, which we solve with cluster-aware validation. Finally, we extend the DML framework beyond the simple partially linear model to handle nonlinear dose-response curves and treatment effect heterogeneity.

2 Wrapping up the Double ML Theory

In the previous lecture, we introduced the core concepts behind Double Machine Learning (DML). We started with the **Partial Linear Model (PLM)**,

$$Y = \theta_0 D + f_0(X) + \epsilon, \quad \mathbb{E}[\epsilon | D, X] = 0$$

where our goal is to estimate the causal parameter θ_0. We saw that a generalization of the Frisch-Waugh-Lovell (FWL) theorem provided a path forward. The FWL theorem gives a natural interpretation of the coefficient on D in the linear regression $\text{OLS}(Y \sim D, X)$: by residualizing, or

partialling out, the effect of X from both Y and D via OLS, we could recover an unbiased estimate of θ_0. This works for fully linear models.

To handle partially linear models we generalized this idea by replacing the OLS in the residualization steps of FWL with flexible Machine Learning models. These new residuals of the treatment and the outcome can be viewed as the true surprises, that cannot be predicted by any function of the controls:

$$\tilde{Y} = Y - \mathbb{E}[Y \mid X] \qquad\qquad \tilde{D} = D - \mathbb{E}[D \mid X]$$

Then the final OLS step in this generalized FWL theorem is (in the population limit) equivalent to solving the corresponding OLS population normal equation, which led to the population estimating equation:[1]

$$\mathbb{E}[(\tilde{Y} - \theta\tilde{D})\,\tilde{D}] = 0, \tag{1}$$

which we argued is **insensitive** to first-stage estimation errors of the nuisance functions.[2]

We identified a critical problem, which we also had to deal with in the doubly robust estimator: using the same data to train the ML models and to estimate θ_0 leads to **overfitting bias**. To eliminate the bias from overfitting, we must use different data samples for learning the nuisance functions and for estimating the main parameter θ_0. Cross-fitting is a data-efficient procedure to accomplish this. This procedure ensures that the predictions for any observation i are generated by a model that was not trained on observation i, thereby preventing overfitting bias. Figure 1 illustrates this process for a single iteration.

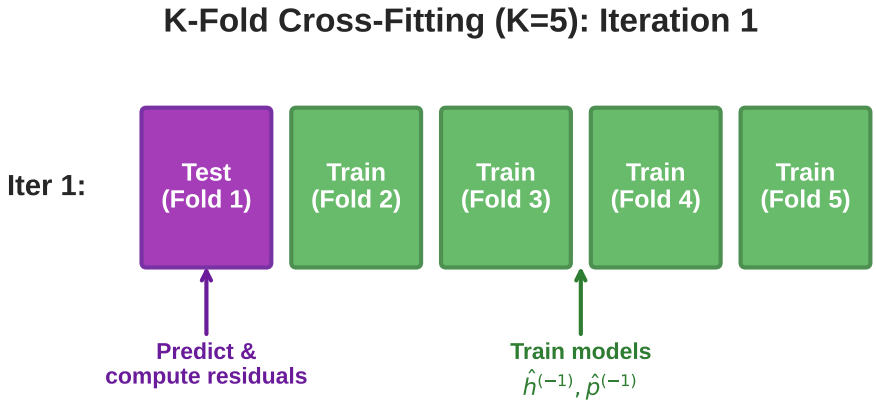

Figure 1: Illustration of the first iteration of 5-fold cross-fitting. The models $\hat{h}^{(-1)}$ and $\hat{p}^{(-1)}$ are trained on data from folds 2 through 5 (green). These models are then used to generate predictions and residuals for the data held out in fold 1 (purple).

[1] Estimating equations that are of the form $\mathbb{E}[m(W;\theta)] = 0$ for some function m, are typically called **moment equations** in the literature.

[2] Such estimating equations (or moment equations) that are insenstive to the first stage nuisance models are called in the literature Neyman-orthogonal moment equations.

All these ingredients led to the final Double ML estimator.

Algorithm: Double ML with K-fold Cross-Fitting

The complete DML algorithm proceeds as follows:

1. Randomly partition the data indices $I = \{1, \dots, n\}$ into K disjoint folds, $I_1, \dots, I_K$.
2. For each fold $k \in \{1, \dots, K\}$:
   (a) Using the data from all other folds (the training set $I \backslash I_k$), train two machine learning models:
   - $\hat{h}^{(-k)}(x)$: A model to predict the outcome Y from the covariates X.
   - $\hat{p}^{(-k)}(x)$: A model to predict the treatment D from the covariates X.

   (b) For each observation i **inside** the test fold I_k, compute the residuals:

$$\check{Y}_i = Y_i - \hat{h}^{(-k)}(X_i) \quad \text{(Surprise in outcome)}$$
$$\check{D}_i = D_i - \hat{p}^{(-k)}(X_i) \quad \text{(Surprise in treatment)}$$

3. After cycling through all K folds, we will have a set of residuals $(\check{Y}_i, \check{D}_i)$ for every observation $i = 1, \dots, n$. Estimate θ_0 by running a simple OLS regression of the outcome residuals on the treatment residuals (without an intercept):

$$\mathbb{E}_n[(\check{Y} - \hat{\theta}\check{D})\,\check{D}] = 0 \quad \Leftrightarrow \quad \hat{\theta} = \frac{\frac{1}{n}\sum_{i=1}^{n} \check{Y}_i \check{D}_i}{\frac{1}{n}\sum_{i=1}^{n} \check{D}_i^2}$$

Python pseudocode for the Double ML algorithm.

```
from sklearn.model_selection import KFold
from sklearn.base import clone
import numpy as np

def double_ml(Y, D, X, ml_model_y, ml_model_d, n_folds=5):
    kf = KFold(n_splits=n_folds, shuffle=True, random_state=42)
    Y_res, D_res = np.zeros(len(Y)), np.zeros(len(D))

    # 1. K-fold cross-fitting loop
    for train_idx, test_idx in kf.split(X):
        # 2a. Train nuisance models on out-of-fold data
        h_model = clone(ml_model_y).fit(X[train_idx], Y[train_idx])
        p_model = clone(ml_model_d).fit(X[train_idx], D[train_idx])
        # 2b. Predict on in-fold data and compute residuals
        Y_res[test_idx] = Y[test_idx] - h_model.predict(X[test_idx])
        D_res[test_idx] = D[test_idx] - p_model.predict(X[test_idx])
    # 3. Final OLS on all residuals
    theta_hat = np.mean(Y_res * D_res) / np.mean(D_res**2)

    return theta_hat, Y_res, D_res
```

2.1 Asymptotic Normality and Inference

The insensitivity property of the estimating equations associated with the DML algorithm allows us to argue that the estimator is asymptotically normally distributed, enabling us to construct confidence intervals and perform hypothesis tests. This limit normal distribution is the exact analogue of the limit normal distribution that we saw for the coefficient in an OLS estimator. The only difference is that now the linearly predicted residuals are replaces by the flexibly predicted residuals.

Theorem: Asymptotic Normality of DML Estimator

Under certain regularity conditions on the data generating process and the convergence rates of the nuisance estimators, the cross-fitted DML estimator $\hat{\theta}$ is asymptotically normal:

$$\sqrt{n}(\hat{\theta} - \theta_0) \xrightarrow{d} N\left(0, \frac{\mathbb{E}[(\tilde{Y} - \theta_0 \tilde{D})^2 \tilde{D}^2]}{(\text{Var}(\tilde{D}))^2}\right)$$

where $\tilde{D} = D - \mathbb{E}[D|X]$ is the true treatment residual and $\tilde{Y} = Y - \mathbb{E}[Y|X]$ is the true outcome residual.

This is a remarkable result. It states that even though we are using potentially very complex, high-dimensional ML models ($\hat{h}$ and $\hat{p}$), the final estimate $\hat{\theta}$ behaves like a simple OLS estimator, albeit with residuals that correspond to "true" surprises in the outcome and the treatment and not just linearly un-predictable surprises. The key is that the first-stage estimation errors from the ML models do not affect the asymptotic distribution of $\hat{\theta}$—a direct consequence of the insensitivity property of the moment function.

Nuisance Rate Conditions. The theorem requires the nuisance models to be "good enough" in terms of their prediction error, measured by Root Mean Squared Error (RMSE). Specifically, we need:

1. **Product Rate Condition:** The product of the RMSEs of the two nuisance models must vanish faster than the square root of the sample size:

$$\sqrt{n} \cdot \text{RMSE}(\hat{h}) \cdot \text{RMSE}(\hat{p}) \to 0$$

2. **Treatment Model Rate Condition:** The RMSE of the treatment model must vanish faster than the fourth root of the sample size:

$$n^{1/4} \cdot \text{RMSE}(\hat{p}) \to 0$$

3. **Consistency:** The outcome model must also be consistent:

$$\text{RMSE}(\hat{h}) \to 0$$

As we have seen, many common ML methods such as Random Forests, Lasso, and Neural Networks can satisfy these conditions under assumptions of sparse or low-dimensional structure in the underlying functions.

The Overlap Condition in the PLM

For the variance to be well-defined, we need the denominator $\text{Var}(\tilde{D})$ to be strictly greater than zero. This is the analogue of the overlap (or positivity) assumption in the PLM. It requires that the treatment D is not perfectly predictable by the covariates X. For binary treatments, this condition is $\text{Var}(D - p(X)) = \mathbb{E}[p(X)(1 - p(X))] > 0$, which means the propensity score must not be exactly 0 or 1 for all observations. This is a much weaker condition than the pointwise overlap ($0 < p(x) < 1$ for all x) required by the doubly robust estimator, as it only needs to hold on average over the distribution of X.

2.2 Standard Errors and Confidence Intervals

To perform inference, we need an estimate of the asymptotic variance. We can construct a consistent estimator by plugging in the empirical analogues of the true quantities:

$$\hat{\sigma}^2 = \frac{\mathbb{E}_n[(\check{Y} - \hat{\theta}\check{D})^2\check{D}^2]}{(\mathbb{E}_n[\check{D}^2])^2}$$

The standard error of our DML estimate is then $\text{s.e.}(\hat{\theta}) = \hat{\sigma}/\sqrt{n}$. This allows us to construct a 95% confidence interval as $\hat{\theta} \pm 1.96 \cdot \text{s.e.}(\hat{\theta})$.

Python pseudocode for DML with standard errors.

```
def double_ml_with_se(Y, D, X, ml_model_y, ml_model_d, n_folds=5):
    theta_hat, Y_res, D_res = double_ml(Y, D, X, ml_model_y, ml_model_d, n_folds)

    # Compute standard error from residuals
    epsilon_res = Y_res - theta_hat * D_res
    sigma_sq_hat = np.mean(epsilon_res**2 * D_res**2) / (np.mean(D_res**2)**2)
    se = np.sqrt(sigma_sq_hat / len(Y))

    return theta_hat, se
```

3 Synthetic Example: The Importance of Good Nuisance Models

Let's see the DML algorithm in action in a synthetic data example. The data is generated from a PLM with a true causal effect of $\theta_0 = -1.5$ and a complex, nonlinear confounding function $f_0(X)$.

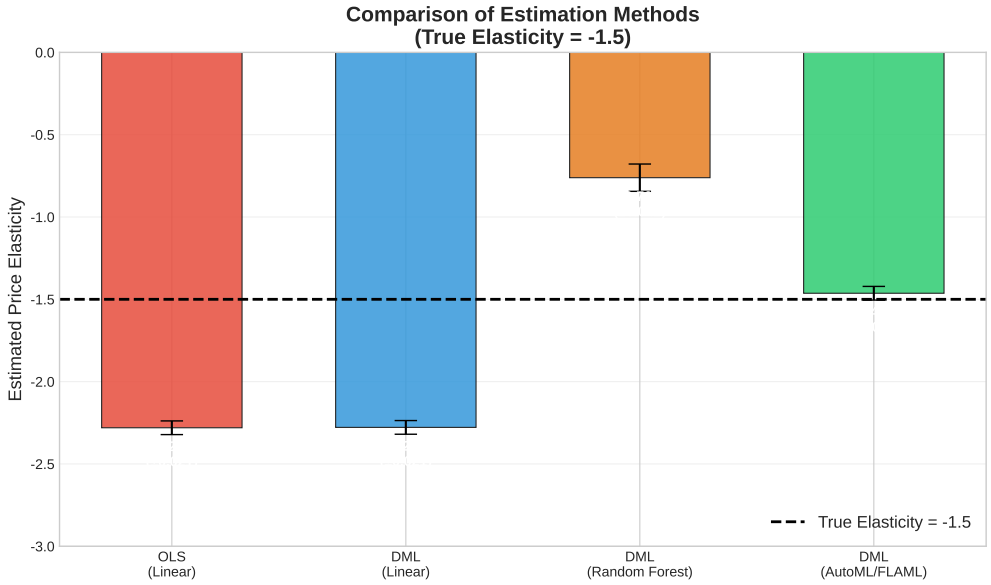

Figure 2: Comparison of price elasticity estimates from three different methods. The true elasticity is -1.5. OLS and DML with linear nuisance models are severely biased due to their inability to capture the nonlinear confounding function $f_0(X)$. In contrast, DML with an auto-tuned Gradient Boosting model (via FLAML) successfully learns the confounding structure and recovers an estimate very close to the true value. The DGP uses $n = 10,000$ samples, and the treatment is also a function of the confounders.

Figure 2 shows the estimates from three models. Naive OLS is severely biased ($\hat{\theta}_{OLS} = -2.280$), as is DML with simple linear regression for the nuisance models ($\hat{\theta}_{DML-LR} = -2.278$). This is a classic case of model misspecification. The linear models are forced to find the best linear approximation of the true nonlinear function $f_0(X)$, but because this approximation error is correlated with the treatment, it leads to omitted variable bias.

The Danger of Un-tuned Models

In our synthetic data example, using a default, un-tuned Random Forest model for the nuisance functions (DML-RF) yields an estimate of $\hat{\theta}_{DML-RF} = -0.761$. This is still far from the true value of -1.5. This highlights a critical lesson: **the quality of the DML estimate depends directly on the quality of the nuisance models.** Simply using an off-the-shelf ML model is not enough. We must ensure the models are well-suited to the data and properly tuned. Using automated machine learning (AutoML) tools like FLAML to search for the best model and its optimal hyperparameters is a powerful strategy to ensure the nuisance models are as accurate as possible, which in turn gives us the best chance of obtaining an unbiased causal estimate.

The Success of Double Machine Learning

By using the best model (Gradient Boosting) and hyperparameters chosen by FLAML, DML can learn a much more accurate approximation of the true confounding function. The DML estimator yields an estimate of $\hat{\theta}_{DML-Auto} = -1.4632$, which is remarkably close to the true value of -1.5. The 95% confidence interval is narrow and comfortably contains the true parameter. Figure 3 provides a visual intuition for why this works. It plots the outcome residuals ($\check{Y}$) against the treatment residuals ($\check{D}$) generated by the DML algorithm. By accurately removing the predictable components from Y and D, the plot reveals the underlying linear relationship between the "surprises" in each variable. The slope of the regression line through these residuals is our DML estimate of the causal effect, which is very close to the true effect.

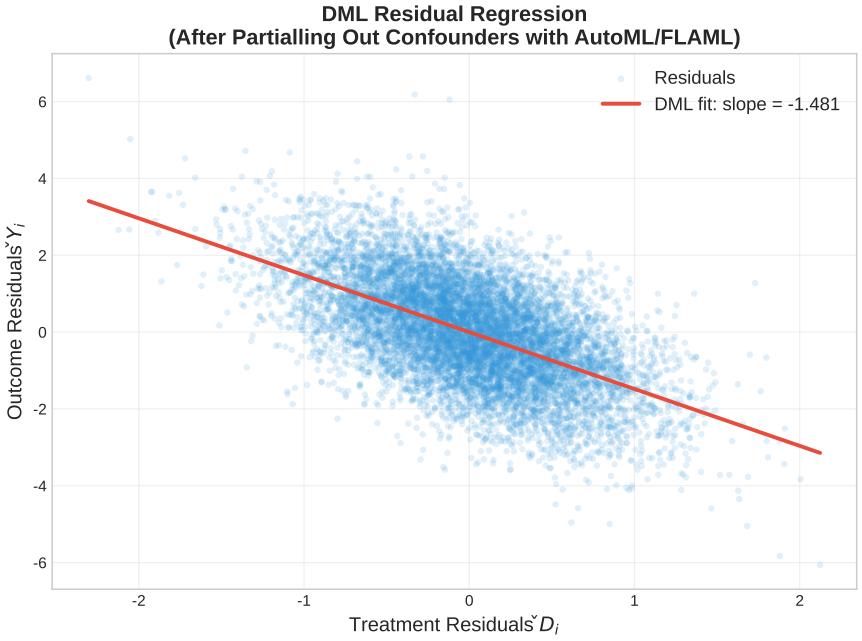

Figure 3: A scatter plot of the outcome residuals against the treatment residuals from a DML model using AutoML for the nuisance models. After partialling out the confounding effects, the simple linear relationship between the residualized variables becomes clear. The slope of this relationship is the DML estimate of the causal effect.

3.1 Using DML Libraries

For convenience, libraries like **EconML** and **DoubleML** automate the DML procedure. In EconML, the 'LinearDML' class allows specifying the ML models for the outcome and treatment. In the DoubleML library, a 'DoubleMLData' object is created, and the 'DoubleMLPLR' (Partially Linear Regression) method is called. Both libraries produce estimates that are in the same ballpark (e.g., -1.4632 with a corresponding standard error), with small deviations due to the randomness

inherent in the K-fold cross-validation splits. Deep down, these libraries are executing the exact same algorithm as the manual implementation.

Using the `econml` library.

```
from econml.dml import LinearDML
# Initialize the DML model with your chosen nuisance models
dml_model = LinearDML(model_y=AutoMLWrapper(), model_t=AutoMLWrapper(), cv=5)
# Fit the model and get the summary
dml_model.fit(Y, D, W=X)
print(dml_model.summary())
```

Using the `doubleml` library.

```
import doubleml as dml
# 1. Create a data object
dml_data = dml.DoubleMLData.from_arrays(x=X, y=Y, d=D)
# 2. Initialize the DML model with your chosen nuisance models
dml_plr = dml.DoubleMLPLR(dml_data, ml_l=AutoMLWrapper(), ml_m=AutoMLWrapper(), n_folds=5)
# 3. Fit the model and get the summary
dml_plr.fit()
print(dml_plr.summary)
```

Practical Note: A FLAML API Quirk

A minor practical issue can arise when using the FLAML (Fast Lightweight AutoML) library. The developers do not return 'self' at the end of the '.fit()' method. This means that chaining calls like 'model.fit(X, y).predict(X)' will fail because '.fit()' returns 'None'. A simple wrapper class can be used to fix this issue, allowing FLAML models to be passed to packages like EconML or DoubleML that expect the standard scikit-learn API.

3.2 Reporting Nuisance Diagnostics

Beyond the point estimate, it is also important to report the accuracy of the nuisance estimators as a diagnostic. The out-of-sample R-squared for both the outcome model ($\hat{h}$) and the treatment model ($\hat{p}$) should be computed and reported. This can be done easily using the residuals from the DML procedure: $R^2 = 1 - \mathbb{E}_n[\text{residual}^2]/\text{Var}(\text{variable})$.

Python pseudocode for DML Nuisance Diagnostics.

```
def double_ml_with_se_and_diagnostics(Y, D, X, ml_model_y, ml_model_d, n_folds=5):
    theta_hat, Y_res, D_res = double_ml(Y, D, X, ml_model_y, ml_model_d, n_folds)
    # Compute standard error from residuals
    epsilon_res = Y_res - theta_hat * D_res
    sigma_sq_hat = np.mean(epsilon_res**2 * D_res**2) / (np.mean(D_res**2)**2)
    se = np.sqrt(sigma_sq_hat / len(Y))
    # Nuisance diagnostics
    r2y, r2d = 1 - np.mean(Y_res**2) / np.var(Y), 1 - np.mean(D_res**2) / np.var(D)
    return theta_hat, se, r2y, r2d
```

For instance, in our synthetic example, the linear model achieves an outcome R-squared of about 0.25, the un-tuned random forest model achieves about 67%, and the AutoML model achieves about 88%. This large difference in predictive accuracy is what allows the better models to estimate the treatment effect correctly. The simpler models leave a lot of explainable variation in both the outcome and the treatment residuals. When the correlation of these imperfect residuals is measured, it captures not only the causal signal but also some bias from the remaining explainable variation that the model was not good enough to capture. Therefore, achieving better and better out-of-sample R-squared values for the nuisance models is always desirable, as it provides greater trust in the resulting causal effect estimate. If someone presents a causal effect estimated with a linear model, and a gradient-boosted model achieves a much better out-of-sample R-squared, the linear model results should be viewed with skepticism.

Table 1 summarizes the results from all the models we ran on the synthetic data. It clearly shows how the estimate for θ improves as the R-squared of the nuisance models increases. The final AutoML-based DML model recovers an estimate that is very close to the true value of -1.5, demonstrating the power of using flexible, well-tuned models for the nuisance components.

Table 1: Summary of Price Elasticity Estimates on Synthetic Data (True $\theta_0 = -1.5$)

Method	**Estimate ($\hat{\theta}$)**	**Std. Error**	**Nuisance R_Y^2**	**Nuisance R_D^2**
OLS (linear controls)	-2.280	0.0212	–	–
Double Machine Learning Implementations:				
DML with Linear Regression	-2.278	0.0212	0.2532	0.5214
DML with Random Forest (un-tuned)	-0.7612	0.0423	0.6756	0.7952
DML with AutoML (FLAML)	-1.4756	0.0213	0.8843	0.8921
Library Implementations (using AutoML models):				
EconML 'LinearDML'	-1.4756	0.0213	–	–
DoubleML 'DoubleMLPLR'	-1.4676	0.0212	–	–

4 DML vs. Doubly Robust: A Tale of Two Insensitive Estimators

Both the Double Machine Learning (DML) estimator and the Doubly Robust (DR) estimator are built on the powerful idea of using an **insensitive or orthogonal moment equation**. This property allows us to use complex machine learning models for nuisance components while still achieving valid statistical inference for our causal parameter of interest. However, they are designed for different settings and target slightly different parameters. Understanding their respective strengths and weaknesses is key to choosing the right tool for your causal inference problem.

Table 2 provides a side-by-side comparison of the key features of the two estimators.

Table 2: Comparison of Doubly Robust (DR) and Double Machine Learning (DML) Estimators

Feature	Doubly Robust (DR)	Double Machine Learning (DML)
Treatment Type	Primarily for **binary** (more generally, **categorical**) treatments.	Handles **continuous** or binary treatments naturally.
Target Estimand	The population Average Treatment Effect (ATE): $\mathbb{E}[Y(1) - Y(0)]$.	Coefficient θ in a PLR Model: $Y = \theta D + f(X) + \epsilon$. Without PLR, it estimates a weighted average effect (e.g., overlap weighted ATE, weighted average derivative).
Key Assumption	Unconfoundedness: $Y(d) \perp D\|X$.	Partial Linearity and Unconfoundedness.
Overlap Requirement	Requires **pointwise** overlap: $0 < P(D = 1\|X) < 1$ for all X.	Requires only **average** overlap: $\text{Var}(D - \mathbb{E}[D\|X]) > 0$.
Key Property	**Double Robustness**: Consistent if either the outcome or propensity model is correct.	**Insensitivity (Neyman Orthogonality)**: First-stage errors don't affect final estimate's distribution.
Variance	Can have high variance in regions where the propensity score $p(X)$ is close to 0 or 1.	Naturally down-weights low-overlap regions, often leading to lower variance.
Efficiency	Semiparametrically efficient for the ATE under no assumptions.	Can have lower variance by assuming PLR.

Rule of Thumb: Which Estimator Should I Use?

- **Use the Doubly Robust (DR) estimator when:**
  - Your treatment is **binary** (more generally **categorical**).
  - Your primary goal is to estimate the population **Average Treatment Effect**.
  - You have good overlap across the entire covariate space.
- **Use the Double Machine Learning (DML) estimator when:**
  - Your treatment is **continuous** (e.g., price, dosage, time spent).
  - The partial linearity assumption (or its generalization presented in Section 6) seems plausible for your application. Or you are ok with weighted average effects or weighted average derivatives for implicitly defined weights (sometimes not-interpretable); see next Section.
  - You are concerned with **robustness to poor overlap** (i.e., extreme propensities).

4.1 What Does DML Estimate When the Model is Misspecified?

An important question is: what does the vanilla DML estimator converge to if the true outcome model is not partially linear? Regardless of whether the true model is partially linear, DML

always estimates the line fit of the residuals: $\mathbb{E}[\tilde{Y}\tilde{D}]/\mathbb{E}[\tilde{D}^2]$. This quantity can be shown to equal a **weighted average treatment effect**, where the weights are proportional to the conditional variance of the treatment given X:

$$\theta_{\text{DML}} = \frac{\mathbb{E}[\tau(X) \cdot \text{Var}(D|X)]}{\mathbb{E}[\text{Var}(D|X)]}$$

For binary treatments, $\text{Var}(D|X) = p(X)(1 - p(X))$. This variance is large when $p(X)$ is close to 0.5 and tiny when $p(X)$ is close to 0 or 1. So DML effectively down-weights groups with extreme propensities (where there is little signal for learning the treatment effect) and up-weights groups where the treatment is more randomized (where there is a lot of signal). It estimates a weighted average treatment effect, putting more weight on regions with a lot of signal. This is why DML has lower variance: it focuses on the parts of the population where the causal effect can be most precisely estimated. However, one should not confuse this with the population ATE; if the true interest is in the unweighted ATE, the DR estimator should be used.

For continuous treatments, the interpretation is more involved. DML estimates a **weighted average derivative**. If the treatment is conditionally Gaussian, this becomes a variance-weighted average derivative, which is interpretable as an overlap-weighted average effect. If the treatment is non-Gaussian, the weights become less interpretable, but they are always positive, meaning DML never assigns negative weight to the derivative at any point. This is a pro for DML: even if all assumptions are dropped, the estimate can still be interpreted as some form of positively weighted average of the causal effect. But if the unweighted ATE is the target, the DR estimator is the appropriate choice.

Case Studies: Choosing the Right Estimator

To solidify our understanding, let's consider a few scenarios:

1. **Scenario 1: Amazon Price Changes.** You are an economist at Amazon, and your manager wants to know the effect of a small price change on sales. Your treatment variable is **price (continuous)**. Which estimator is more natural to use, and why?

   Answer: DML is the natural choice here. The treatment is continuous, which is exactly what the PLM framework is designed for. The DR estimator is built for binary treatments and cannot be directly applied.

2. **Scenario 2: Get-Out-the-Vote Campaign.** You are analyzing a get-out-the-vote campaign. For a certain subgroup of your data (e.g., young voters in a specific state), almost everyone received the treatment (a reminder to vote). You are worried about **poor overlap**. Which estimator might provide a more stable estimate?

   Answer: DML would be more stable. The DR estimator's variance can explode if propensity scores are near 0 or 1. DML's reliance on average overlap rather than pointwise overlap makes it more robust. However, one should be aware that DML will effectively exclude the groups for which the treatment was deterministic (e.g., young voters who all received the treatment) and will only estimate the effect for the groups with more randomized treatment (e.g., older voters). The resulting estimate is the average effect on those groups, not the average effect of the entire campaign.

3. **Scenario 3: Heterogeneous Drug Effects.** You suspect that the effect of a new drug is not constant but **varies significantly** based on patient characteristics (age, gender, etc.). What does the DML estimator give you in this case of heterogeneous effects? How does it differ from the ATE estimated by DR?

   Answer: The basic DML estimator for the PLM assumes a constant treatment effect, θ_0. If the true effect is heterogeneous, DML estimates a specific weighted average of the individual effects (as described above), which is not necessarily the same as the population ATE that the DR estimator targets. To handle this properly, one would need to move to the Generalized PLM (discussed in Section 6) and include interaction terms between the treatment and the covariates.

5 Application: DML for Demand Estimation with Panel Data

We now move to a practical, real-world application of Double Machine Learning: estimating the price elasticity of demand for products sold on Amazon. This case study, based on the work of Bach et al. (2024), will force us to confront several challenges that arise when working with real world data. The main problem that we will focus here is that we typically have panel data (data

that follows the same entities over time).

5.1 The Data and The Goal

Our dataset consists of approximately 38,000 observations for 3,800 unique toy car products (identified by their ASIN) on Amazon, tracked over roughly one year. This is a **panel dataset**, as we have multiple observations over time for each product.

- **Outcome (Y):** `Q_t`, the log of the inverse sales rank. This serves as a proxy for the log of the quantity sold, as lower sales rank implies higher sales.
- **Treatment (D):** `P_bb_t`, the log of the Buy Box price.
- **Controls (X):** A rich set of control variables, including lagged quantity and price, time dummies (to capture seasonality), and high-dimensional text embeddings from the product descriptions.

Our goal is to estimate the **price elasticity of demand**, which in this log-log specification is simply the coefficient θ_0 on the log-price variable.

5.2 Challenge 1: Endogeneity of Price, Controlling for Lags, Images and Text

A naive OLS regression of log quantity on log price is likely to be severely biased. Unobserved, persistent product characteristics, such as **brand quality**, can affect both the price a seller sets and the demand from consumers. High-quality products might command higher prices but also have higher baseline demand. This creates positive confounding, which would bias our elasticity estimate towards zero (or even make it positive).

Lagged quantity and price variables serve as important proxies for the unobserved, persistent product quality. A product that sold well last period (high lagged quantity) is likely a high-quality product. By including these lags in the regression, we are controlling for a significant portion of the confounding effect of quality. By doing so, we see a dramatic change in the coefficient from -0.11 (severely biased towards zero) in the naive OLS regression of quantity on price, to -0.69 (more plausible) once lagged variables are included as controls. This demonstrates the critical importance of controlling for confounders.

On top of lagged variables, one might also want to incorporate the extra information contained in the text and images. However, running a gradient-boosted forest or FLAML directly on a raw image would produce poor results. The correct approach is to extract embeddings from the image or text. If a deep neural network, such as a text transformer, is used to predict the treatment or the outcome from the text, it implicitly creates an embedding and then runs a linear (or penalized linear) model on that embedding. Similarly, a convolutional neural network (CNN) could be used for images. The DML framework provides a natural template: two models need to be trained (one for the outcome, one for the treatment), and ML intuition guides the choice of model architecture (CNNs for images, transformers for text).

In practice, with only about 30,000 data points, training a transformer from scratch is not reasonable. Instead, pre-trained transformers (like LLaMA for text) and pre-trained image neural networks can be fine-tuned on the specific task of predicting demand and price. The authors of the paper did exactly this: they took pre-trained models, jointly fine-tuned a multimodal neural network to predict both price and demand simultaneously, and then used the embedding layer of the fine-tuned neural network as the features X in a downstream DML algorithm. Implicitly, this entire process is a DML algorithm, because the fine-tuning of the transformer is itself the training of a complex ML model. Even if OLS is the best thing to do after the embedding construction, the overall procedure is a full-fledged DML algorithm because the first stage of building the embedding is the part of building a flexible ML predictive model.

The authors performed this fine-tuning on a training set, and the data we are using for this analysis loads the validation data (a separate half of the data not used for fine-tuning). The resulting embeddings were also subjected to dimensionality reduction using a **Johnson-Lindenstrauss transform**, which takes a high-dimensional vector (e.g., 1,000 dimensions) and projects it into a lower-dimensional space (e.g., 250 dimensions) while approximately preserving all pairwise distances among samples. This makes it feasible to run DML with a reasonably sized set of features. The authors also performed K-means clustering on the embeddings to create five product groups based on product characteristics, which can be used to explore treatment effect heterogeneity across product clusters.

5.3 Challenge 2: Clustered Errors

In a panel dataset, observations for the same product across different time periods are not independent. Unobserved factors specific to a product (like its brand reputation or a latent quality factor) will persist over time, creating correlation in the error terms for that product. If we ignore this correlation, standard OLS standard errors will be artificially small, leading to overconfidence in our estimates and potentially false discoveries.

The Clustered Sandwich Estimator

The solution is to use a **cluster-robust** variance estimator. Let g index the clusters (products) and i index the observations within a cluster. The standard error formula from Section 2.2 assumes each observation is independent. The clustered version modifies this by first summing the relevant quantities within each cluster before averaging across clusters. Let $\psi_i = (\check{Y}_i - \hat{\theta}\check{D}_i)\check{D}_i$ be the score for observation i. The non-clustered variance estimator is proportional to $\sum_i \psi_i^2$. The clustered estimator, in contrast, is proportional to $\sum_g (\sum_{i \in g} \psi_i)^2$. It allows for arbitrary correlation between observations *within* a cluster, but assumes independence *across* clusters. The full formula for the variance of $\hat{\theta}$ is:

$$\hat{\sigma}^2_{cluster} = \frac{1}{(\mathbb{E}_n[\check{D}^2])^2} \frac{1}{n^2} \sum_{g=1}^{G} \left(\sum_{i \in g} (\check{Y}_i - \hat{\theta}\check{D}_i)\check{D}_i \right)^2$$

where G is the number of clusters and n is the total number of observations. This formula correctly accounts for the intra-cluster correlation, providing valid standard errors for inference.

Discussion: The Need for Clustered Standard Errors

What would be the problem if we don't use clustered standard errors?

Answer: If we fail to cluster our standard errors at the product level, we are implicitly assuming that all 38,000 observations are independent. This is clearly false. The result is that our standard errors would be far too small, leading to artificially narrow confidence intervals and inflated t-statistics. We would be wildly overconfident in our results and would likely commit a Type I error (falsely rejecting the null hypothesis). Clustering is essential for valid inference in panel data.

5.4 Challenge 3: Data Leakage in Cross-Fitting

The final and most subtle challenge arises when we combine cross-fitting with panel data. A standard `KFold` cross-validation splits observations randomly. In our panel setting, this means that different observations (from different weeks) for the *same product* could end up in both the training set and the test set for a given fold.

This is a form of **data leakage**. The nuisance models can effectively "cheat." When training on data for Product A from weeks 1-10, the model can learn the product-specific average demand. When it is then asked to predict demand for Product A in week 11 (which is in the test set), it can use this leaked information, leading to an artificially good prediction. This results in residuals $(\check{Y}_i, \check{D}_i)$ that are too small, which in turn biases the final DML estimate and invalidates our inference.

Solution: Cluster-Aware Cross-Fitting

We must use a **cluster-aware cross-fitting** strategy. Instead of splitting individual observations, we split entire groups (clusters) of observations. In our case, we split by product (ASIN). This ensures that all data for a given product belongs to exactly one fold. The `GroupKFold` function in scikit-learn is designed for this purpose.

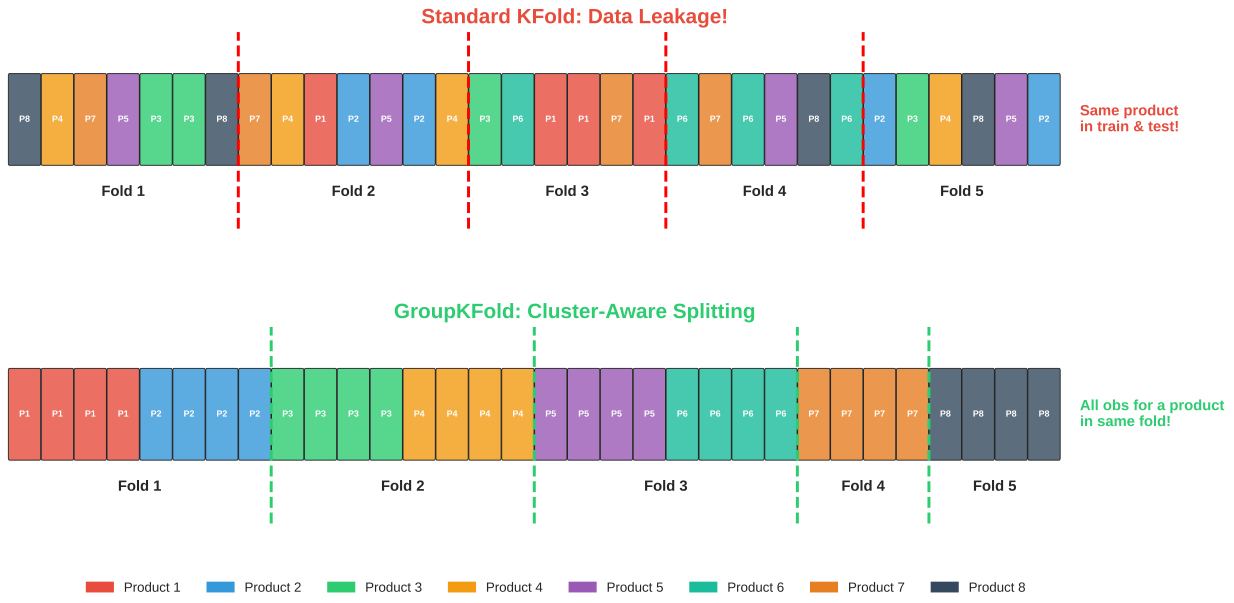

Figure 4: Comparison of standard KFold and GroupKFold for panel data. Standard KFold randomly assigns observations, leading to data leakage where the same product appears in both training and testing folds. GroupKFold correctly keeps all observations for a single product within the same fold, preventing leakage.

Discussion: The Peril of Standard KFold

Why do we use ‘GroupKFold‘? What would be the problem with using standard ‘KFold‘ for cross-fitting in this panel data setting?

Answer: ‘GroupKFold‘ ensures that all observations from the same group (product/ASIN) are in the same fold. This means when we predict for product ‘i‘, we never trained on any observations from product ‘i‘. Standard ‘KFold‘ would violate this, leading to data leakage, overfitting of the nuisance functions, and biased final treatment effect estimates.

5.5 Putting It All Together: DML with Clustering

The correct DML procedure for this panel data problem combines these solutions. Moreover, including the time period as a feature in the set of controls allows the predictive models for the treatment and outcome to be functions of time, effectively capturing time-fixed effects.

Algorithm: DML for Panel Data

1. Split the data into K folds using `GroupKFold`, with the product ID (ASIN) as the grouping variable.
2. For each fold k:
   - Train nuisance models $\hat{h}^{(-k)}$ and $\hat{p}^{(-k)}$ on the out-of-fold data.
   - Compute residuals $(\check{Y}_i, \check{D}_i)$ for all observations i in the test fold k.
3. Run a final OLS regression of $\check{Y}$ on $\check{D}$ to get $\hat{\theta}$.
4. Compute the standard error for $\hat{\theta}$ using a **cluster-robust** variance-covariance matrix estimator for the final OLS regression.

Manual implementation of DML for panel data

```
# 1. Loop with GroupKFold to ensure cluster-aware splitting
for train_idx, test_idx in GroupKFold(n_splits=5).split(X, Y, groups=df['ASIN']):
    # 2. Train nuisance models on train data
    model_y.fit(X.iloc[train_idx], Y.iloc[train_idx])
    model_d.fit(X.iloc[train_idx], D.iloc[train_idx])

    # 3. Predict on test data and get residuals
    Y_resid.iloc[test_idx] = Y.iloc[test_idx] - model_y.predict(X.iloc[test_idx])
    D_resid.iloc[test_idx] = D.iloc[test_idx] - model_d.predict(X.iloc[test_idx])

# 4. Final OLS on residuals with clustered standard errors
dml_est = sm.OLS(Y_resid, D_resid).fit(
    cov_type='cluster', cov_kwds={'groups': df['ASIN']})
```

While implementing this manually is instructive, professional libraries like `DoubleML` and `EconML` handle these details automatically. When using these libraries, one simply needs to specify the cluster variable in the data object or fit method.

Using the `doubleml` library with panel data

```
import doubleml as dml
from lightgbm import LGBMRegressor

# 1. Create DoubleMLData object, specifying the cluster variable
dml_data = dml.DoubleMLData(df, y_col='Q_t', d_cols='P_bb_t',
                            x_cols=control_vars, cluster_cols='ASIN')
# 2. Initialize and fit the DML model
# The library automatically uses GroupKFold due to cluster_cols
ml_l = LGBMRegressor()
ml_m = LGBMRegressor()
dml_plr = dml.DoubleMLPLR(dml_data, ml_l=ml_l, ml_m=ml_m)
dml_plr.fit()

# 3. The summary provides cluster-robust standard errors by default
print(dml_plr.summary)
```

Using the `econml` library with panel data

```
from econml.dml import LinearDML
from sklearn.linear_model import LinearRegression

# The ‘groups‘ argument in fit() enables cluster-robust inference
econml_dml = LinearDML(model_y=LGBMRegressor(),
                       model_t=LGBMRegressor(),
                       cv=5)
# The library automatically uses GroupKFold due to groups being provided
econml_dml.fit(df[outcome], df[treatment], W=df[control_cols], groups=df['ASIN'])

# 3. The summary provides cluster-robust standard errors by default when groups are provided
print(econml_dml.summary())
```

By correctly applying this procedure, we obtain a price elasticity estimate of ≈ -0.7. This estimate is economically plausible and statistically significant.

6 Beyond PLM: Generalized Models and Heterogeneous Effects

The partially linear model, $Y = \theta D + f(X) + \epsilon$ relies on a key assumption: the treatment effect is linear and constant. That is, a one-unit increase in D has the same effect on Y, regardless of the initial level of D or the values of the covariates X. In many real-world scenarios, this is too restrictive.

- The effect of a drug may saturate at high doses (nonlinear dose-response).
- The effect of a price change may be larger for low-income individuals than for high-income individuals (treatment effect heterogeneity).

The DML framework can be extended to handle these cases by moving to a **Generalized Partially Linear Model (GPLM)**.

$$Y = \theta_0' \phi(D, X) + f_0(X) + \epsilon$$

Here, instead of a single treatment D, we have a vector of features $\phi(D, X)$ that are constructed from the treatment and covariates. Our goal is to estimate the vector of coefficients θ_0. This flexible form allows for model nonlinearities and interactions that capture treatment effect heterogeneity.

Examples of Basis Functions $\phi(D, X)$.

- **Polynomial Basis:** To model a nonlinear dose-response curve, we can use polynomial features of the treatment: $\phi(D, X) = (D, D^2, \dots, D^p)'$. This allows the conditional expectation of the outcome to be a polynomial function of the dose: $\mathbb{E}[Y|D = d, X = x] = \theta_1 d + \theta_2 d^2 + \cdots + \theta_p d^p + f_0(x)$.
- **Interaction Basis:** To model treatment effect heterogeneity, we can interact the treatment with covariates: $\phi(D, X) = (D, D \cdot X_1, D \cdot X_2, \dots)'$. Here, the treatment effect, $(\theta_1 + \theta_2 X_1 + \theta_3 X_2 + \dots)$, is now a function of the covariates.

6.1 DML for the GPLM

The DML algorithm extends naturally to the GPLM. The core idea remains the same, but we must now residualize the new feature vector $\phi(D, X)$ before running the final regression.

Algorithm: DML for the Generalized PLM

1. For each fold k, train models $\hat{h}^{(-k)}(X)$ to predict Y from X and $\hat{q}_j^{(-k)}(X)$ to predict each component $\phi_j(D, X)$ of the feature vector from X.
2. Compute residuals for the outcome $\check{Y}_i = Y_i - \hat{h}^{(-k)}(X_i)$ and for each feature component $\check{\phi}_{i,j} = \phi_j(D_i, X_i) - \hat{q}_j^{(-k)}(X_i)$.
3. Run a multivariate OLS regression of the outcome residuals $\check{Y}$ on the feature residuals $\check{\phi}$ to get the coefficient vector estimate $\hat{\theta}$.

$$\hat{\theta} = (\mathbb{E}_n[\check{\phi}\check{\phi}'])^{-1}\mathbb{E}_n[\check{\phi}\check{Y}]$$

Special Case: Separable Features. A common and computationally convenient case is when the basis functions are separable, i.e., $\phi(d, x) = \psi(x) \cdot d$. This is a model-based approach to treatment effect heterogeneity, as we are hard-coding the functional form of the heterogeneous effect. In later lectures, we will examine more flexible methods for treatment effect heterogeneity. In this case, we only need to estimate one nuisance model for the treatment, $\hat{p}(x) = \mathbb{E}[D|X = x]$, and the residual for the feature vector becomes $\check{\phi}_i = \psi(X_i) \cdot (D_i - \hat{p}^{(-k)}(X_i))$. This is much more efficient than fitting a separate ML model for each component of ϕ.

6.2 Inference for the GPLM

Inference proceeds similarly to the basic DML case. The estimator $\hat{\theta}$ is asymptotically normal, $\sqrt{n}(\hat{\theta} - \theta_0) \xrightarrow{d} N(0, \Sigma)$, with a covariance matrix Σ estimated using a sandwich formula based on the residuals:

$$\hat{\Sigma} = (\mathbb{E}_n[\check{\phi}\check{\phi}'])^{-1}\mathbb{E}_n[\check{\phi}\check{\phi}'\hat{\epsilon}^2](\mathbb{E}_n[\check{\phi}\check{\phi}'])^{-1}$$

where $\hat{\epsilon}_i = \check{Y}_i - \hat{\theta}'\check{\phi}_i$. This is the standard HC0 robust covariance matrix that packages like `statsmodels` calculate. From this, we can derive confidence intervals for various quantities of interest.

Inference on Conditional Dose-Response Curve / CATE

Our goal is to construct a confidence interval for the Conditional Average Treatment Effect,

$$\tau(d, x) = \mathbb{E}[Y(d) - Y(0)|X = x], \tag{CATE}$$

at a specific dose d and covariate value x. Under the GPLM, this is simply:

$$\tau(d, x) = \theta_0'(\phi(d, x) - \phi(0, x)).$$

We can estimate this with

$$\widehat{\tau}(d, x) = \hat{\theta}'(\phi(d, x) - \phi(0, x)).$$

Note that:

$$\sqrt{n}(\widehat{\tau}(d, x) - \tau(d, x)) = \sqrt{n}\delta_{d,x}'(\hat{\theta} - \theta_0)$$

were $\delta_{d,x} = \phi(d, x) - \phi(0, x)$ Thus, since $\sqrt{n}(\hat{\theta} - \theta_0))$ is approximately distributed as a Gaussian $N(0, \Sigma)$, we get that the error in the CATE is approximately distributed as the inner product of such a Gaussian with the vector $\delta_{d,x}$, which is a Gaussian with variance $\delta_{d,x}'\Sigma\delta_{d,x}$. Hence, we can construct a 95% CI as:

$$\hat{\theta}'\delta_{d,x} \pm 1.96 \cdot \sqrt{\frac{\delta_{d,x}'\hat{\Sigma}\delta_{d,x}}{n}}$$

Inference on Average Dose-Response Curve / ATE

Our goal is to construct a CI for the Average Treatment Effect,

$$\tau(d) = \mathbb{E}[Y(d) - Y(0)]. \tag{ATE}$$

Under the GPLM model, this is

$$\tau(d) = \theta_0'\mathbb{E}[\phi(d, X) - \phi(0, X)].$$

We estimate this with $\widehat{\tau}(d) = \hat{\theta}'\bar{\delta}_d$, where $\bar{\delta}_d = \mathbb{E}_n[\phi(d, X_i) - \phi(0, X_i)]$. Here, there are two sources of uncertainty: estimating θ_0 and estimating $\mathbb{E}[\phi]$. The full variance formula includes both terms. The 95% CI is:

$$\hat{\theta}'\bar{\delta}_d \pm 1.96 \cdot \sqrt{\frac{\bar{\delta}_d'\hat{\Sigma}\bar{\delta}_d + \mathrm{Var}_n(\hat{\theta}'\delta_{d,X_i})}{n}}$$

Inference on Marginal Effects

We can also perform inference on marginal effects. The conditional marginal effect is $\text{ME}(d, x) = \theta_0' \frac{\partial \phi(d,x)}{\partial d}$. The Average Marginal Effect (AME) is $\text{AME} = \theta_0' \mathbb{E}[\frac{\partial \phi(D,X)}{\partial D}]$. Confidence intervals for both can be constructed analogous to the CATE and ATE, simply redefining the vector $\delta_{d,x}$ as $\frac{\partial \phi(d,x)}{\partial d}$ and in the AME case using an average vector $\bar{\delta} = \mathbb{E}_n[\delta_{D,X}]$ and a variance of $\text{Var}_n(\hat{\theta}' \delta_{D_i,X_i})$.

7 Summary: Four Key Takeaways

This lecture provided a comprehensive overview of Double Machine Learning, from its theoretical foundations to its practical application.

1. **The Full DML Algorithm:** We formalized the DML algorithm, emphasizing the crucial role of **cross-fitting** to eliminate overfitting bias. The resulting estimator is asymptotically normal, which allows for valid statistical inference (confidence intervals and hypothesis testing).

2. **DML vs. Doubly Robust:** We contrasted DML with the DR estimator, highlighting their respective strengths. DML is the natural choice for **continuous treatments** and is more robust to **violations of overlap**. Even when the PLM assumption does not hold, the quantity being estimated is still an interpretable, positively weighted average causal effect. DR is tailored for estimating the **ATE** of binary treatments and is semiparametrically efficient.

3. **DML for Panel Data:** We tackled the practical challenges of applying DML to panel data. This requires two key adjustments: **cluster-aware cross-fitting** (e.g., 'GroupKFold') to prevent data leakage, and **cluster-robust standard errors** to account for correlated observations within groups.

4. **Generalized DML:** We extended the DML framework to the Generalized PLM, allowing for the estimation of **nonlinear dose-response curves** and **heterogeneous treatment effects** by using basis function expansions. This flexible approach allows the incorporation of text, image, and other rich features into the causal inference framework simply by using pre-trained neural networks as featurizers.

By separating prediction from inference and leveraging the power of modern machine learning in a theoretically sound way, DML provides a powerful and reliable framework for causal inference in complex observational settings.

8 References

- Chernozhukov, V., Chetverikov, D., Demirer, M., Duflo, E., Hansen, C., Newey, W., & Robins, J. (2018). Double/debiased machine learning for treatment and structural parameters. *The Econometrics Journal*, 21(1), C1-C68.

- Bach, P., Chernozhukov, V., Klaassen, S., Spindler, M., Teichert-Kluge, J., & Vijaykumar, S. (2024). Adventures in Demand Analysis Using AI. *Working Paper*.

- Bach, P., Chernozhukov, V., Kurz, M. S., and Spindler, M. (2022). DoubleML: An Object-Oriented Implementation of Double Machine Learning in Python. *Journal of Machine Learning Research*, 23(53), 1-6.